Tema 2.1:

Embeddings y búsqueda semántica

Minería de textos, Grado en Inteligencia Artificial (GRIA), Universidad de Vigo

© Prof. David Candal Ventureira - 2026

Tabla de contenidos

1 Fundamentos

2 Word embeddings

3 Embeddings textuales

4 Recuperación de información

5 Despliegue

1

Fundamentos

Fundamentos: Representación vs significado

Humanos y máquinas vemos las palabras de manera diferente:

Humanos (semántica):

- caballo == 
- rayas == 
- caballo + rayas == 

- caballo ≈ cebra
- caballo ≠ cabello

Máquinas (representación):

- “caballo” == 99 97 98 97 108 108 111 (ASCII)
- “rayas” == 114 97 121 97 115 (ASCII)
- “caballo” + “rayas” == ��������

- caballo ≠ cebra
- caballo ≈ cabello

Fundamentos: Representación vs significado

Sinónimos y palabras relacionadas se tratan (tokenizan) como **términos independientes**:

	caballo	potro	yegua	corcel	Caballo
doc1	4	0	0	0	1
doc2	0	5	0	0	0
doc3	0	0	6	0	0
doc4	0	0	0	4	0
doc5	2	1	1	1	0

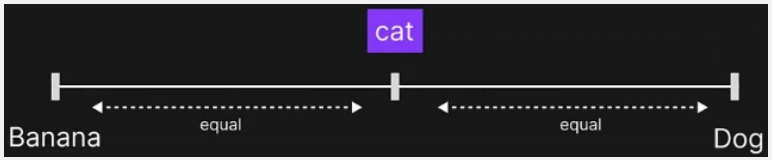
Fundamentos: Representación vs significado

Sinónimos y palabras relacionadas se tratan (tokenizan) como **términos independientes**:

Escenario ideal:



Realidad:



Fundamentos: La relevancia del contexto

- El **contexto nos da información** relevante acerca de cada palabra (incluso cuando se desconoce la palabra).
- Ejemplo:
  - *Nunca salgo de casa sin mi _____ porque con este tiempo puede _____ en cualquier momento.*

- "*You shall know a word by the company it keeps*" (J.R. Firth, 1957) → “Conocerás una palabra por la compañía que frecuenta”.
- Las palabras que **aparecen en los mismos entornos lingüísticos (contextos)** tienden a **tener significados similares**.

2

Word embeddings

Word embeddings: Estáticos (Word2Vec)

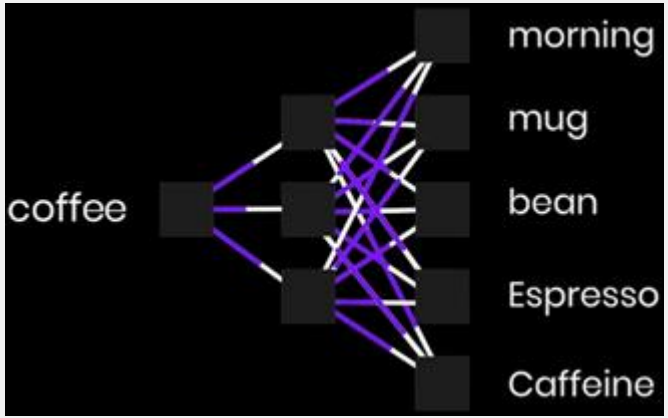
- **Word2Vec** (Mikolov, 2013) aplica la hipótesis distribucional de Firth mediante una red neuronal con aprendizaje auto-supervisado, para **construir la información semántica (significado) de cada palabra**.
- Construcción del modelo Word2Vec (Skip-gram):
  1. Construimos y entrenamos un modelo neuronal para **predecir palabras que suelan formar parte del contexto** de una palabra dada.
  2. Cada entrada es un vector *one-hot* (1 en el índice de la palabra, 0 en el resto) de tamaño V (tamaño del vocabulario). La salida (del modelo completo) es un vector de tamaño V con las **probabilidades de cada palabra del vocabulario de pertenecer al contexto** de la palabra de entrada.
  3. El modelo se entrena para **pares de palabra de entrada – vector de contexto positivo + contexto negativo** (n palabras que suelen aparecer + n palabras aleatorias).

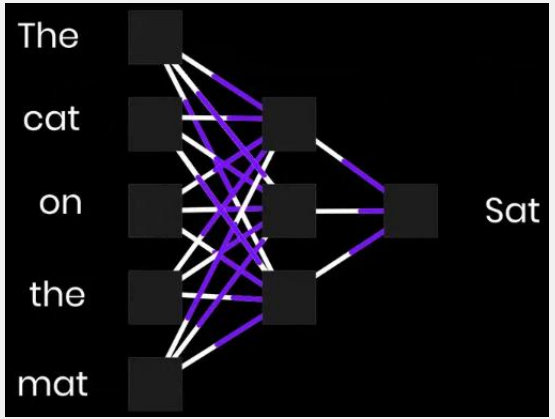
Word embeddings: Estáticos (Word2Vec)

- **Word2Vec** (Mikolov, 2013) aplica la hipótesis distribucional de Firth mediante una red neuronal con aprendizaje auto-supervisado, para **construir la información semántica (significado) de cada palabra**.
- Construcción del modelo Word2Vec (Skip-gram):
  4. La **capa oculta** $W \in \mathbb{R}^{VxD}$ transforma el vector de entrada en un vector de tamaño D que **aproxima el significado de la palabra**. La capa de salida $W' \in \mathbb{R}^{DxV}$ transforma este vector de tamaño D a un vector de tamaño V con las **probabilidades de cada palabra del vocabulario de aparecer en el contexto** de la palabra de entrada.
  5. Se elimina la salida y **nos quedamos únicamente con la "capa oculta"**, que genera un vector de tamaño D (embedding).

Word embeddings: Estáticos (Word2Vec)

- Dos estrategias:
  - **Skip-gram**: La entrada es una palabra, la salida una estimación de su contexto.
  - **CBOW**: La entrada es un contexto, la salida es una estimación de la palabra que genera ese contexto.





- En ambos casos, el modelo de embeddings se queda con la capa oculta (1ra capa) y cada fila de la matriz W se corresponde con el vector de embeddings de una palabra.

Word embeddings: Estáticos (Word2Vec)

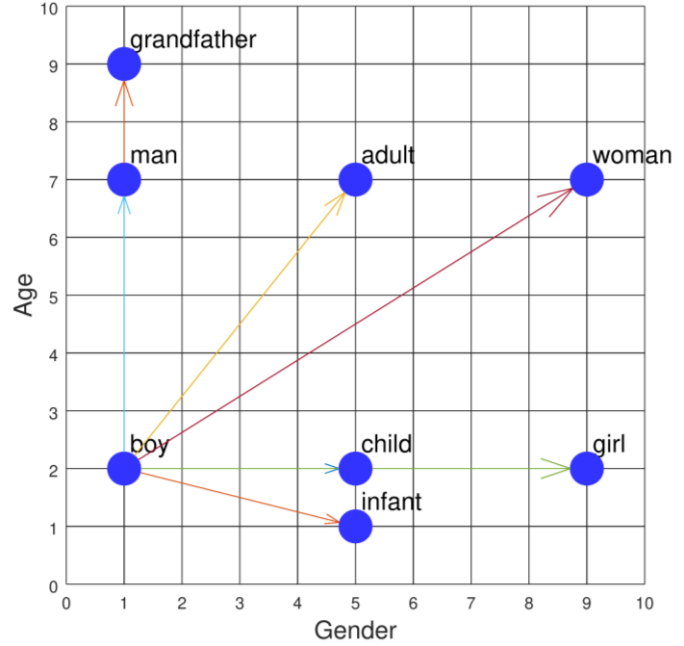
- Cada posición del vector de embeddings (**dimensión**) **almacena el grado de relación de la palabra con un cierto atributo semántico latente**, identificado automáticamente por el modelo (no predefinido ni generalmente interpretable).





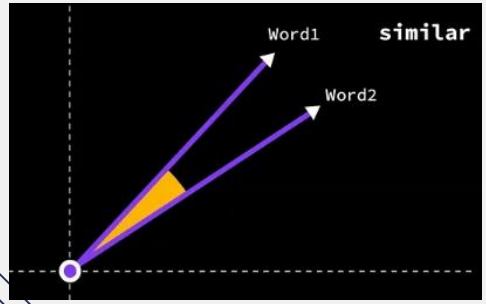
Word embeddings: Estáticos (Word2Vec)

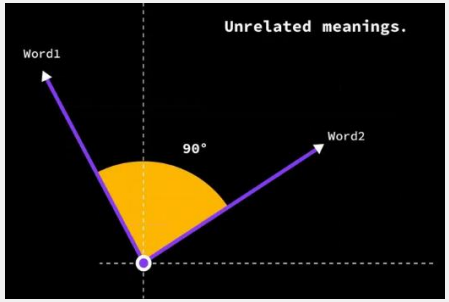
- Álgebra lineal semántica:
  - grandfather – boy + girl = **grandmother**
  - man – adult + child = **boy**
  - king – man + woman = **queen**

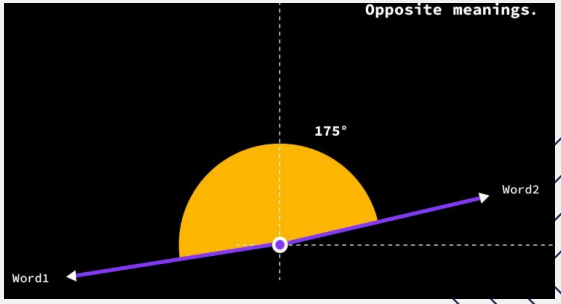


Word embeddings: Estáticos (Word2Vec)

- La similitud semántica entre dos palabras se mide a través de la **similitud coseno** de sus embeddings.
- La similitud coseno **compara el ángulo entre los vectores** de dos términos (no tiene en cuenta su magnitud):
  - Palabras similares -> Similitud coseno ≈ 1 (≈ 0º).
  - Palabras opuestas -> Similitud coseno ≈ -1 (≈ 180º).
  - Palabras no relacionadas -> Similitud coseno ≈ 0 (≈ 90º).
- La **similitud coseno es igual al producto escalar** (producto punto) de dos **vectores normalizados** (de magnitud 1).







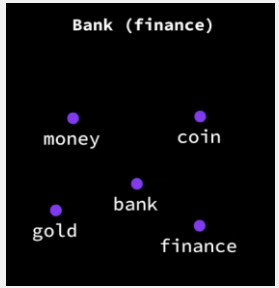
Word embeddings: Estáticos (Word2Vec)

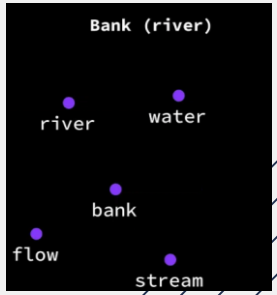
- Los word embeddings estáticos parten de una representación BoW:
  - 1 término -> 1 vector (significado).
  - Son **incapaces de adaptarse a la polisemia** de los términos (múltiples significados dependientes del contexto).
- Ej: “banco” puede significar “asiento” o “negocio”.

- Los **embeddings estáticos** de palabras polisémicas calculados mediante Word2Vec, en general, serán el resultado de **la suma o promedio de las dimensiones principales de cada significado**.
  - El embedding “almacena” (promedia) todos los significados de la palabra, pero aumenta el ángulo (su distancia semántica) con cada uno de ellos.
  - Las palabras polisémicas tienen una similitud menor a sus sinónimos directos que las palabras que sólo tienen un significado.

Word embeddings: Contextuales (BERT)

- Las redes neuronales basadas en Transformers como BERT (Bidirectional Encoder Representations from Transformers) solucionan el principal problema de los Word embeddings estáticos.
- En lugar de utilizar el mismo vector estático para representar una misma palabra, **BERT toma como entrada un bloque de texto** y genera, para cada palabra, un **embedding contextual**, que captura el significado de las palabras **en su contexto** (hipótesis de Firth):
  - El embedding de una palabra es **diferente para cada uno de sus significados**.
  - El embedding de una palabra es **diferente para un mismo significado en función de su contexto**.

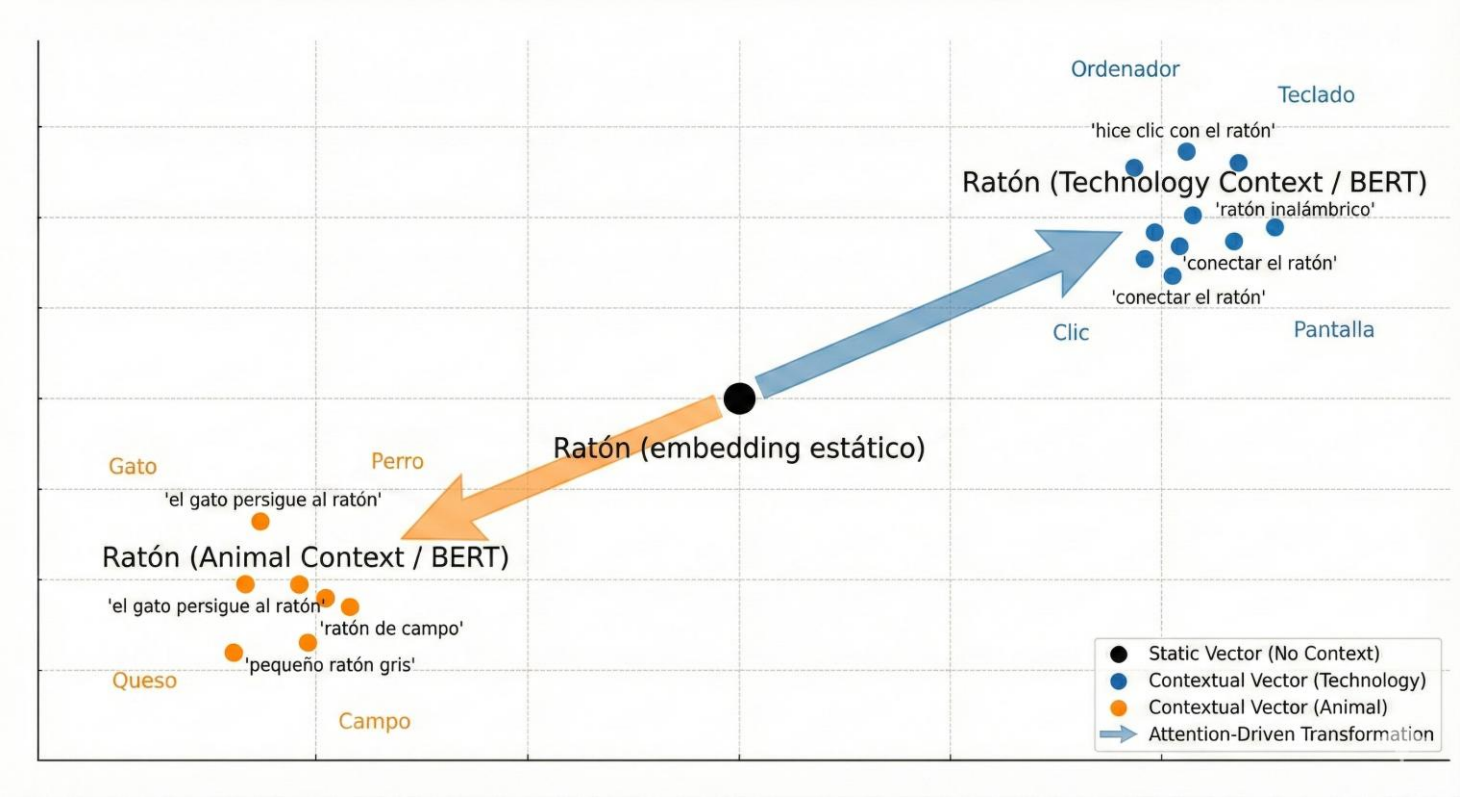




Word embeddings: Contextuales (BERT)

- **Entrenamiento**: Busca **predecir una palabra en base a su contexto** (como CBOW), buscando el **significado** de la palabra (no sólo su representación). Se construye:
  - Una matriz de embeddings estáticos (similar a Word2Vec).
  - Matrices de atención, que representan cómo debe calcularse la relevancia de una palabra frente a las palabras en el contexto.
- **Inferencia**: Dado un texto de entrada:
  - Se obtiene el embedding estático de cada palabra del texto.
  - Se calcula la relevancia de cada palabra con las palabras de su contexto.
  - Se realiza la media del embedding estático inicial y de las palabras de su contexto, ponderada por su relevancia (atención).
  - Se repite este ciclo para cada capa del modelo (BERT-base -> 12 capas).

Word embeddings: Contextuales (BERT)



3

Embeddings textuales

Embeddings textuales: Composición de embeddings

- Los word embeddings nos permiten **comparar tokens por su significado**.
- ¿Cómo podemos trabajar sobre el significado de un conjunto de tokens? (frase, documento, etc).
  - Principio de composicionalidad: El **orden de los tokens importa** (“El perro muerde al hombre” ≠ "El hombre muerde al perro“) -> No podemos simplemente promediar o sumar los vectores
  - Los modelos de ML deben trabajar con **matrices de tamaño fijo** -> Los embeddings de texto (sentence embeddings o document embeddings) son vectores de tamaño fijo sin importar el número de tokens de entrada.



Embeddings textuales: Sentence-BERT (SBERT)

- **SBERT** parte de la arquitectura original de BERT y la entrena para generar embeddings que **proyecten el significado de un texto completo**.
- El modelo SBERT se entrena como sigue:
  1. Se toman dos textos como entrada.
  2. Se generan los embeddings contextuales de cada palabra para cada texto.
  3. Para cada texto, se combinan los embeddings contextuales para generar un embedding único.
  4. Se comparan los embeddings de ambos textos. Dos textos similares deben tener direcciones similares, dos textos opuestos deben tener direcciones opuestas, y dos textos independientes deberían tener direcciones perpendiculares.
  5. Los pesos del modelo se ajustan durante el entrenamiento para que los vectores resultantes se comporten de la manera descrita anteriormente.

Embeddings textuales: Sentence-BERT (SBERT)

- **Arquitectura siamesa**:
  - Conceptualmente, SBERT se comporta como **dos modelos de tipo BERT** (con los mismos pesos).
  - “Cada uno” toma un texto de entrada y genera un embedding contextual para cada token.
  - **Pooling**: Los embeddings de cada token **se combinan en un vector único** que almacena el significado del texto completo.
  - Estos embeddings se comparan para actualizar los pesos de “ambos” modelos BERT simultáneamente.
- **Estrategias de pooling**:
  - **Mean pooling** (estándar): Calcula la **media de todos los embeddings contextuales** de token.
  - **Max pooling**: **Para cada dimensión, toma el valor máximo** de todos los embeddings contextuales de token.

Embeddings textuales: Sentence-BERT (SBERT)

- Durante su entrenamiento, SBERT toma:
  - **Pares de puntuación continua**:
    - Se toman dos frases, junto con un **nivel numérico de similitud** entre las frases, generado por un humano:
    - Ejemplos:
      - ["El niño juega", "El niño se divierte", 0.9].
      - [“La sopa está fría", “La sopa está caliente", 0.6].
      - ["El niño juega", “La sopa está fría", 0.05].
  - **Pares de inferencia lógica**:
    - Se toman dos frases y una **etiqueta de vinculación semántica** (implicación, contradicción o neutral):
    - Ejemplos:
      - ["Un perro corre por el parque", "Un animal se está moviendo", Implicación].
      - ["Un perro corre por el parque", "Un niño lee un libro", Neutral].
      - [“La sopa está fría", “La sopa está caliente", Contradicción].

Embeddings textuales: Sentence-BERT (SBERT)

- SBERT consigue comprimir el contexto de un texto. Captura adecuadamente la semántica del texto, pero es **incapaz de diferenciar correctamente algunas situaciones contradictorias**.
- Ejemplo:
  - La sopa que he pedido está muy fría.
  - La sopa que he pedido está muy caliente.
- Aunque sus significados son opuestos, **las palabras compartidas entre ambas frases tienen un peso muy importante en el embedding contextual final**.
- Ejemplos de expresiones problemáticas: atributos/estados físicos (frío/calor, lleno/vacío), sentimiento y opinión (bueno/malo), navegación (izquierda/derecha), cuantificadores (siempre/nunca), transiciones (comprar/vender), negaciones (introducir un no en una frase).

Embeddings textuales: Modelos cross-encoder

- Los modelos **cross-encoder** parten de la arquitectura de BERT y se entrenan para capturar de manera fidedigna la similitud entre dos textos:
  - En lugar de procesar los textos de manera independiente, **se procesan ambos cada vez**:
    - [CLS] TEXTO_A [SEP] TEXTO_B [SEP]
  - Al procesarse juntos, **todas las palabras de ambos textos** interactúan a lo largo de las capas del modelo.
  - La etiqueta [CLS], sin valor semántico inicial, absorbe la información de la interacción entre todas las palabras.
  - El modelo cross-encoder añade una **capa final**, que convierte el embedding de [CLS] a un resultado numérico que refleja la **similitud de las dos frases**.

Embeddings textuales: Modelos cross-encoder

- Durante el entrenamiento:
  - El modelo cross-encoder parte de **pares preetiquetados de similitud** (2 textos + valor de similitud) o **inferencia lógica** (2 textos + etiqueta de similitud).
  - El modelo ajusta los pesos tanto de la capa BERT como de la capa resumen, corrigiendo el error entre su predicción y la etiqueta de entrada.
- Dependiendo de cómo se diseñe la capa final, el modelo puede devolver:
  - La **probabilidad de similitud**: Valor [0,1] que representa la similitud entre las dos frases (1 = totalmente similar, 0 = totalmente opuesto o neutro).
  - **Probabilidades de inferencia lógica**: Tres probabilidades independientes cuya suma es el 100%, para señalar el grado de **implicación, neutralidad o contradicción**.

Embeddings textuales: SBERT vs cross-encoder

- Diferencias entre SBERT y cross-encoder:
  - **SBERT genera embeddings** -> Permite **precalcular los vectores por adelantado para su consulta** (similitud coseno) en producción.
  - **Cross-encoder** toma cada par de textos para **generar su similitud** -> La inferencia debe realizarse en producción -> Mayor coste y latencia.
- Aplicaciones: Recuperación y re-clasificación (retrieve & re-rank):
  - **Fase 1**: Usamos **SBERT** para búsqueda rápida de documentos relevantes para una consulta. Los embeddings de los documentos ya están precalculados, sólo debemos calcular el embedding de la query y el cálculo de similitud coseno con los embeddings de los documentos.
  - **Fase 2**: Usamos **cross-encoder** para ordenar la relevancia entre los *n* documentos clasificados como similares.

Embeddings textuales: Multilingüismo

- Para poder tratar como similares dos textos en diferentes idiomas, es necesario ajustar el modelo SBERT mediante **destilación de conocimiento**:
  1. Tomamos como base un modelo **mBERT**, multilenguaje, adaptado al vocabulario de diferentes idiomas.
  2. Añadimos al modelo mBERT una **capa de pooling**, que permita generar un único embedding textual a partir de los word embeddings de cada token del texto, convirtiéndolo en un modelo SBERT multilingüe.
  3. Preparamos un **corpus de pares de frases**, donde una de las frases está en inglés y la otra en uno de los idiomas a los que queremos dar soporte en nuestro nuevo modelo (incluido el inglés).
  4. Pasamos la **frase en inglés** por el modelo **SBERT** y **su par por el modelo SBERT multilingüe**.
  5. **Ajustamos los pesos del modelo SBERT multilingüe** siguiendo una función de pérdida que penalice la diferencia entre el vector generado y aquel generado por el modelo SBERT.

Embeddings textuales: Multilingüismo

- Para poder tratar como similares dos textos en diferentes idiomas, es necesario ajustar el modelo SBERT mediante **destilación de conocimiento**.
- Ejemplo:

Texto	Embedding
"El gato está sobre la mesa"	[0.12, -0.45, 0.88...]
"The cat is on the table"	[0.11, -0.46, 0.89...]
"Le chat est sur la table"	[0.12, -0.44, 0.88...]

Embeddings textuales: Adaptación de dominio

- Un modelo SBERT general puede ofrecer malos resultados cuando opera bajo un dominio específico:
  - **Contexto implícito**: Existe información contextual que pueda no ser incluida explícitamente en el texto. Ejemplo: “La carga es baja” (¿peso? ¿batería? ¿resistencia? ¿saturación? …).
  - **Precisión terminológica**: Existen palabras que en el dominio general parecen similares, pero pueden ser opuestas en un dominio específico. Ejemplo: Virus y bacteria.
  - **Relevancia de la jerga**: Palabras que tienden a ser poco relevantes en el dominio general pueden serlo dentro de un dominio específico. Ejemplo: Tipo de tornillo en un manual de ingeniería.

Embeddings textuales: Adaptación de dominio

- Adaptación de un modelo SBERT genérico a un dominio específico:
  1. Nos apoyamos en un modelo LLM para generar de manera sintética pares pregunta – respuesta vinculados al dominio:
     - Pares con respuestas positivas (texto o documento).
     - Pares con respuestas negativas (texto o documento).
     - Buscamos pares negativos que, en el dominio general, sean semánticamente similares a la pregunta, pero no la respondan correctamente.
  2. Realizamos fine-tuning del modelo SBERT, ajustando los pesos originales del modelo para que maximice la similitud con los pares positivos y las minimice con los negativos.

4

Recuperación de información

Recuperación de información: Búsqueda semántica vs léxica

- **Búsqueda léxica (BM25)**: Búsqueda por ocurrencia de palabras.
- **Búsqueda semántica (SBERT)**: Búsqueda por representaciones semánticas.

- Ejemplo:
  - Query: “Cómo cerrar mi cuenta”.
  - B. léxica:
    - Doc. recuperado: “Instrucciones sobre cómo mantener la cuenta abierta al cerrar la aplicación en el móvil”.
    - Ocurrencia de todas las palabras, documento no relevante.
  - B. semántica:
    - Doc. recuperado: “Guía para la cancelación definitiva de servicios y baja de usuario“.
    - A pesar de no incluir ninguna de las palabras de la query, el documento es muy relevante.

Recuperación de información: Búsqueda semántica

- Los modelos SBERT destinados a recuperación de información utilizan un mecanismo de entrenamiento simplificado:
  - El corpus (pequeño y homogéneo) dificulta el ajuste preciso por similitud.
  - No buscan un valor de similitud preciso (ej: 0.9).
  - Buscan simplemente aislar los documentos no relevantes para una query dada, incluso aunque compartan términos.

- **Entrenamiento para IR (triplet loss)**:
  - Utilizamos una LLM para **generar queries** (que incluyan o no términos que ocurren en el documento) **para cada documento**. Utilizamos **BM25 para encontrar**, en base a esa query, **documentos por ocurrencia de términos**, de los que **eliminamos** a través de un LLM los documentos que pudieran ser **relevantes**.
  - Durante el entrenamiento, para cada terna (query – positivo – negativo), buscamos que la **similitud de la query con el documento positivo sea mayor que la similitud con el documento negativo, con al menos un margen α**.
  - Los pesos **se mantienen** si el **documento positivo supera en similitud al documento negativo** por, al menos, un **margen α**, y **se ajustan en caso contrario**.

Recuperación de información: Búsqueda semántica

- Los modelos semánticos actuales de IR se basan en un algoritmo más eficiente y escalable, conocido como **Multiple Negatives Ranking Loss (MNRL)**:
  - Con triplet loss, sólo generamos **un documento negativo por cada documento positivo**.
  - Solución:
    - Considerar un batch de pares query – documento positivo (ej: 1024) y **suponer que los documentos positivos de las otras queries**, por estadística, **son negativos para la query correspondiente**.
    - Generamos una matriz NxN (ej: 1024 x 1024) para entrenar calculando **N queries de manera simultánea**, con **1 documento positivo y N-1 negativos por query**.

Recuperación de información: Búsqueda semántica

- A la hora de realizar la consulta sobre el corpus:
  - **KNN (k-nearest neighbors)**: Calculamos la similitud coseno con los embeddings de **todos los documentos**, ordenamos el resultado y **entregamos los k con mayor similitud** (ej: 10M de documentos).
  - **ANN (approximate nearest neighbors) basado en clústering**:
    - Previamente, **dividimos el corpus en C clústeres de documentos por similitud**, calculando el centro geométrico de cada grupo. Pueden existir **múltiples niveles** de clasificación.
    - Realizamos la **consulta contra los vectores promedio de cada grupo**, recorriendo los niveles hasta llegar al clúster más similar.
    - Ej: “motor”. La consulta inicial nos devuelve el clúster de ingeniería. Calculamos la similitud contra los documentos de este grupo, evitando los documentos del resto (química, medicina…).

Recuperación de información: Búsqueda semántica

- A la hora de realizar la consulta sobre el corpus:
  - **ANN basado en grafos, HNSW (hierarchical navigable small world)**:
    - Previamente, **construimos un grafo** (red telaraña), donde se determinan por niveles **documentos "representantes"** de los documentos de nivel inferior.
    - En cada nivel, **se evalúa la similitud con los documentos representantes**. Se selecciona el documento más similar y se baja al siguiente nivel. Los representantes del nuevo nivel serán el documento seleccionado en el nivel anterior y **documentos cada vez más afines al documento seleccionado**.
    - Al llegar al nivel inferior, **se evalúan los documentos alcanzados a través del camino con mayor similitud**, y se ordenan para su retorno como respuesta a la consulta.
    - Al contrario que ANN basado en clústering, **HNSW no tiene umbrales estrictos** (los documentos de menor nivel pueden ser alcanzados a través de múltiples caminos).
  - Tanto ANN como HNSW mantienen en caché los k caminos más similares, a fin de evitar un mínimo local.

Recuperación de información: Búsqueda híbrida

- **Problema de la búsqueda léxica**: Basada únicamente en ocurrencia de términos. No consigue recuperar los documentos semánticamente significativos que no contienen los términos de la búsqueda.
- **Problema de la búsqueda semántica**: Capaces de recuperar documentos por semántica, aunque no incluyan los términos de la búsqueda. Fallan a la hora de obtener búsquedas por identificadores (términos exactos).
    - Ejemplo: La búsqueda semántica de ”Error HTTP 404” puede devolver documentos sobre otros errores HTTP (ambos serían errores de conexión WEB).

- **Búsqueda híbrida (ej: reciprocal rank fusión, RRF):** Se ejecutan ambos tipos de búsquedas en paralelo. Se busca **potenciar los documentos que aparecen en posiciones altas del ranking en ambas búsquedas**, y se penalizan aquellas que aparecen abajo en una de ellas.

$$RRF_{score} = \left(W_{lex}\ \frac{1}{k + Rank_{lex}}\right) + \left(W_{sem}\ \frac{1}{k + Rank_{sem}}\right)$$

Recuperación de información: Métricas de evaluación en IR

- Los buscadores de IR se evalúan a través de métricas que miden el grado en el que son capaces de recuperar y clasificar documentos relevantes dentro de las primeras posiciones del resultado:
  - **Recall@K [Precision@K] (exhaustividad [precisión] en el top K)**: Proporción de documentos relevantes **entre los K primeros** (sobre el corpus o sobre el tamaño de la lista de resultados).

$$Recall@K = \frac{\#\ relevantes\ recuperados}{\#\ relevantes\ en\ corpus} \qquad Precision@K = \frac{\#\ relevantes\ recuperados}{\#\ respuestas}$$

  - **MRR (mean reciprocal rank)**: Mide la **posición que ocupa el primer documento relevante** entre los resultados.

$$MRR = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{rank_i}$$

Recuperación de información: Métricas de evaluación en IR

- Los buscadores de IR se evalúan a través de métricas que miden el grado en el que son capaces de recuperar y clasificar documentos relevantes dentro de las primeras posiciones del resultado:
  - **NDCG@K (normalized discounted cumulative gain)**: Considera la relevancia de los documentos en la lista de resultados, **penalizando logarítmicamente cada posición que bajan los documentos "imprescindibles" de los resultados**.

$$DCG@K = \sum_{i=1}^{k} \frac{rel_i}{log_2(i+1)} \qquad NDCG@K = \frac{DCG@K}{IDCG@K}$$

  - rel_i: Relevancia del documento recuperado en el índice *i* (ej: 0: no relevante, 1: relevante, 2: muy relevante).
  - *IDCG@K*: *DCG@K* ideal (si los documentos estuviesen ordenados por relevancia).

5

Despliegue

Despliegue: Motores de búsqueda semántica

- Los sistemas de búsqueda semántica utilizan **bases de datos vectoriales**:
  - Indexan previamente los documentos (SBERT).
  - Recuperan los documentos calculando el embedding de la query y utilizando ANN para obtener los k más relevantes.

- Ecosistema de DBs vectoriales:
  - **ChromaDB**: Estándar para prototipado.
  - **Qdrant / Milvus**: Mayor eficiencia, orientadas a búsqueda masiva.
  - **Weaviate / OpenSearch**: Búsqueda híbrida.
  - **FAISS / ScaNN**: Librerías que implementan la indexación y recuperación para búsqueda semántica.

Despliegue: Motores de búsqueda semántica

- Los sistemas de búsqueda semántica utilizan bases de datos vectoriales:

```python
import chromadb
from chromadb.utils import embedding_functions
import json


cliente = chromadb.Client()
funcion_multilingue = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="paraphrase-multilingual-MiniLM-L12-v2"
)
coleccion = cliente.create_collection(
    name="mi_coleccion",
    embedding_function=funcion_multilingue,
#    metadata={"hnsw:space": "cosine"}
)


# Fase inicial: Indexación (SBERT) y estructuración (HNSW).
coleccion.add(
    documents=["El perro duerme",
               "El gato maulla",
               "ChromaDB es una DB vectorial",
               "ChromaDB realiza una búsqueda semántica por embeddings"],
    ids=["doc1", "doc2", "doc3", "doc4"]
)

# Los embeddings almacenados en la DB son
datos_almacenados = coleccion.get(
    include=["documents", "embeddings"]
)
for i in range(len(datos_almacenados['ids'])):
    id_doc = datos_almacenados['ids'][i]
    texto = datos_almacenados['documents'][i]
    vector = datos_almacenados['embeddings'][i]
    print(f"Documento: {id_doc}.   Texto: '{texto}'.   Embedding: [{', '.join(f'{x:.4f}' for x in vector[:3])}, ...]")
```

```python
# Búsqueda semántica en el espacio vectorial
query = "Mascota"
query = "Animales"
resultados = coleccion.query(
    query_texts=[query],
    n_results=3,
    include=["documents", "distances", "embeddings"]
)

# Resultados
embedding_query = coleccion._embedding_function([query])[0]
print(f"Query: '{query}'.   Embedding: [{', '.join(f'{x:.4f}' for x in embedding_query[:3])}, ...]")

print("Resultado")
resultados["embeddings"][0] = [
    f"[{', '.join(f'{float(x):.4f}' for x in v[:3])}, ...]"
    for v in resultados["embeddings"][0]
]
print(json.dumps(resultados, indent=4))
```

Despliegue: Motores de búsqueda semántica

```
Documento: doc1.   Texto: 'El perro duerme'.   Embedding: [0.1111, -0.0709, -0.2060, ...]
Documento: doc2.   Texto: 'El gato maulla'.   Embedding: [0.4283, -0.1598, 0.0273, ...]
Documento: doc3.   Texto: 'ChromaDB es una DB vectorial'.   Embedding: [-0.2578, 0.0399, -0.4640, ...]
Documento: doc4.   Texto: 'ChromaDB realiza una búsqueda semántica por embeddings'.   Embedding: [-0.2547, -0.0084, -0.1422, ...]
Query: 'Animales'.   Embedding: [0.0695, -0.0050, -0.1052, ...]
Resultado
{
    "ids": [
        [
            "doc1",
            "doc2",
            "doc3"
        ]
    ],
    "embeddings": [
        [
            "[0.1111, -0.0709, -0.2060, ...]",
            "[0.4283, -0.1598, 0.0273, ...]",
            "[-0.2578, 0.0399, -0.4640, ...]"
        ]
    ],
    "documents": [
        [
            "El perro duerme",
            "El gato maulla",
            "ChromaDB es una DB vectorial"
        ]
    ],
    "uris": null,
    "included": [
        "documents",
        "distances",
        "embeddings"
    ],
    "data": null,
    "metadatas": null,
    "distances": [
        [
            0.5836044549942017,
            0.6032397747039795,
            0.8530624508857727
        ]
    ]
}
```

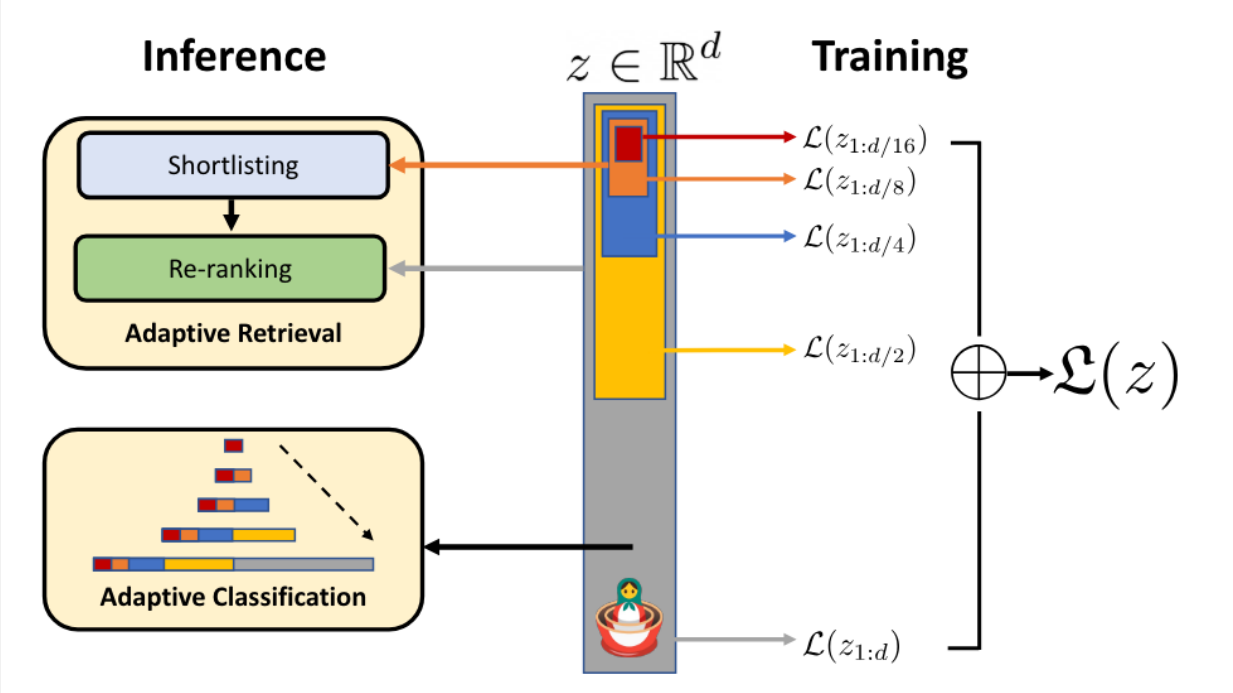
Despliegue: Optimización Matryoshka representation learning (MRL)

- Generalmente trabajamos con un **número masivo de textos** (ej: 1.000 M), cada uno vinculado a un embedding, un **vector de gran tamaño** (ej: 768 dimensiones).
- Aunque se trata de vectores densos, la información semántica relevante se concentra en pocas dimensiones… pero no conocemos a priori cuáles son las más relevantes para cada embedding.

- ¿Podemos comparar embeddings utilizando únicamente las dimensiones más relevantes? -> **Matryoshka representation learning (MRL)**:
  1. Partimos de un modelo de tipo BERT.
  2. A la hora de entrenar el modelo, aplicamos una función de pérdida que, en lugar de utilizar todas las dimensiones de las palabras de los dos textos de entrada, utiliza la **suma de la función de pérdida calculada sobre segmentos progresivamente más extensos de los vectores (ej: 8D, 16D, 64D, …. 768D)**.
  3. De esta forma, el modelo aprende a generar embeddings que permiten ser clasificados semánticamente **usando las primeras posiciones del vector**.

Despliegue: Optimización Matryoshka representation learning (MRL)

- Ejemplo de MRL:

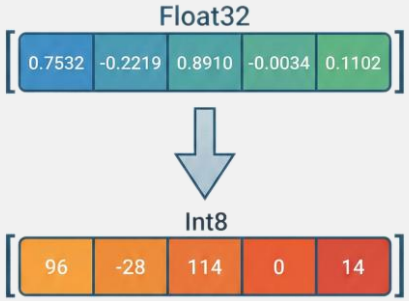


Despliegue: Cuantización de modelos BERT

- Las DB vectoriales almacenan la representación vectorial de cada texto indexado -> Vectores más grandes ocupan más disco/memoria.
- El coste computacional de las operaciones es mucho mayor al usar formatos en coma flotante (Float32) que al utilizar números enteros (Int8).

- La **cuantización** reduce la precisión de los valores de los embeddings:
  - Reduce su tamaño en memoria.
  - Reduce el coste/tiempo de cómputo de las operaciones.

- Ejemplos:
  - Float32 -> Int8: 1/4 espacio en memoria, x2-4 velocidad de operación, 99% de precisión.
  - Float32 -> binario: 1/32 espacio en memoria, x15-30 velocidad de operación, 90% de precisión.

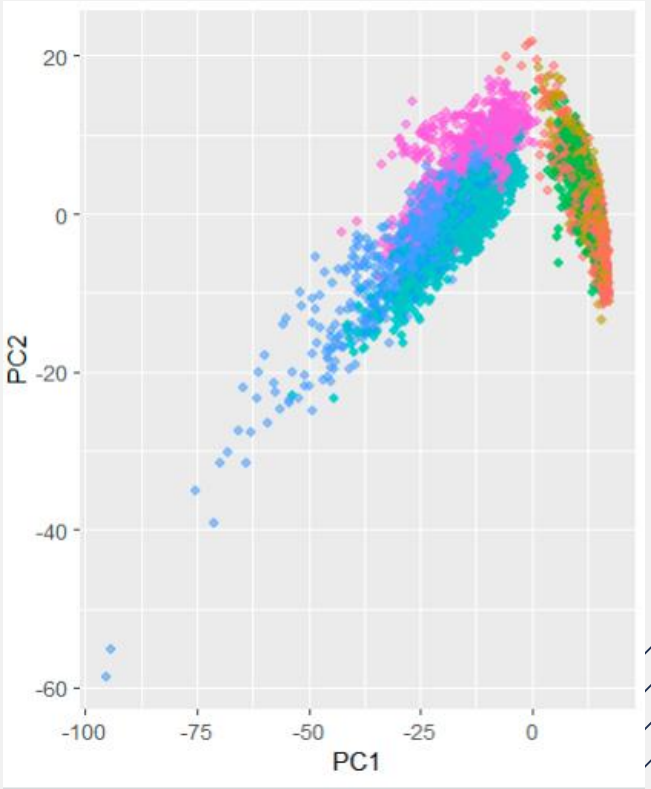


Despliegue: Visualización de los embeddings

- Los embeddings tienen múltiples dimensiones (ej: 768D). Nuestra visión sólo permite procesar 2/3 dimensiones -> **Necesitamos reducir la dimensionalidad a 2 o 3 dimensiones**.
- Técnicas:
  - **PCA (Principal Component Analysis)**: Proyección lineal de la representación multidimensional. Detecta las 2/3 dimensiones con mayor varianza y representa la posición de los datos desde esas proyecciones.
  - **t-SNE (t-Distributed Stochastic Neighbor Embedding)**: Calcula la probabilidad de que cada par de puntos sean vecinos y los representa en 2/3 dimensiones. La distancia entre puntos/clústeres no representa la similitud entre términos. Algoritmo computacionalmente pesado.
  - **UMAP (Uniform Manifold Approximation and Projection)**: Construye un grafo con relaciones entre cada punto. La distancia entre nodos y entre clústeres representa su similitud. Mucho más rápido que t-SNE.

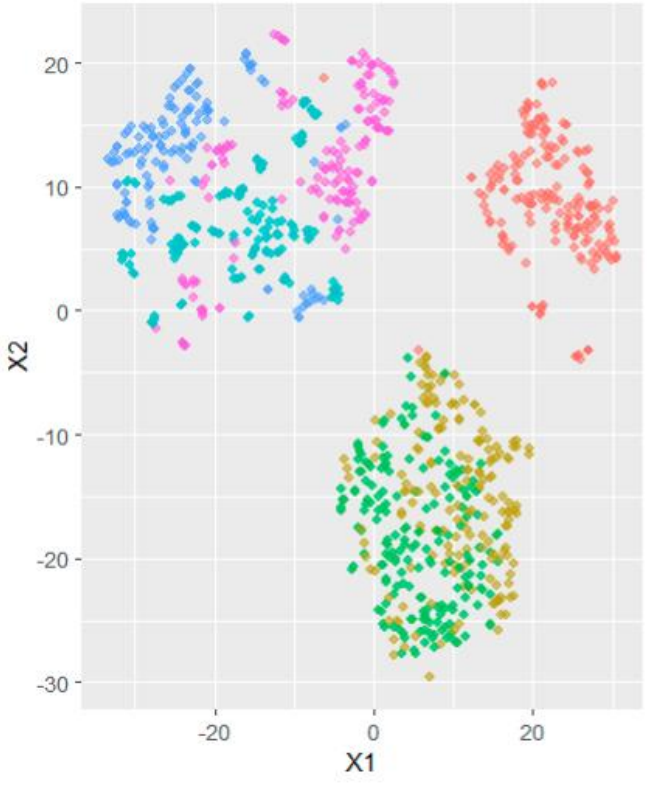
Despliegue: Visualización de los embeddings

- **PCA**:
  - Realiza una **proyección matemática lineal hacia 2/3 ejes** (de mayor varianza).
  - Se pierde la estructura no lineal del hiperespacio original:
    - La posición de cada punto es sólo la proyección del vector en los ejes definidos.
    - No se aprecian bien los clústeres debido al “aplastamiento” de las dimensiones.



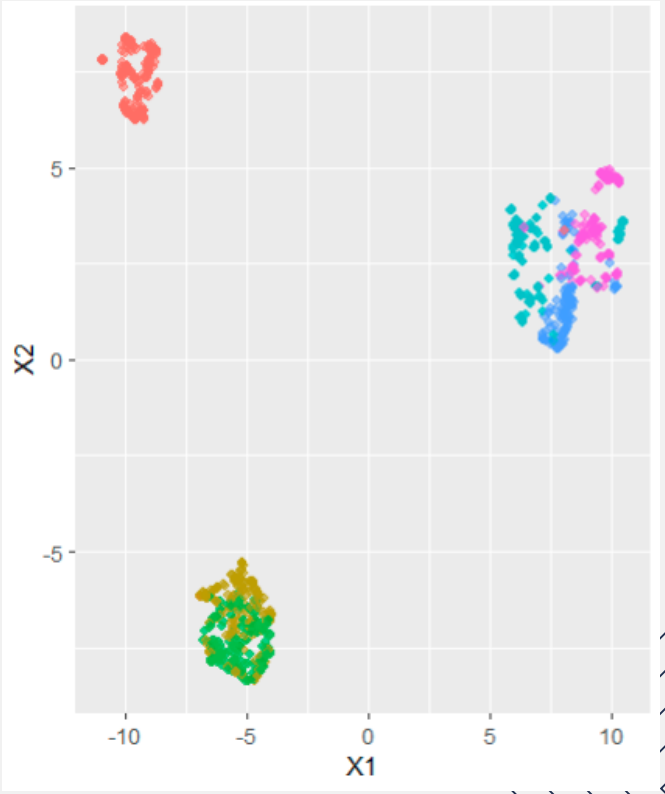
Despliegue: Visualización de los embeddings

- **t-SNE**:
  - Agrupa los vectores en clústeres **calculando la probabilidad de que cada par sean vecinos cercanos**.
  - Consigue separar muy bien los clústeres (categorías).
  - Posiciona cada clúster en una zona del gráfico, pero **sus coordenadas exactas (ejes X e Y) no tienen significado semántico**.
  - **La distancia visual entre clústeres es arbitraria y no mide su similitud real**. La distancia visual entre los puntos dentro de un mismo clúster sí representa su similitud semántica.



Despliegue: Visualización de los embeddings

- **UMAP**:
  - Construye un **grafo topológico conectando a los vecinos más cercanos**, y lo proyecta optimizando su forma.
  - Consigue separar los clústeres con alta precisión y es **computacionalmente mucho más rápido y escalable que t-SNE**.
  - La **distancia entre clústeres sí refleja su similitud**. La distancia visual entre los puntos dentro de un clúster sigue representando su similitud semántica.
  - Sus coordenadas absolutas (ejes X e Y) siguen sin tener un significado semántico, pero **preserva la estructura global del espacio vectorial original**.



Despliegue: Visualización de los embeddings

¿Para qué utilizamos las técnicas de visualización?

- **Depuración del modelo**: Comprobar visualmente si nuestro modelo ha entendido nuestro dominio (ej. ver si agrupa correctamente la jerga técnica de la empresa).
- **Análisis exploratorio de datos (EDA)**: Detectar *outliers*, documentos sucios, texto en otros idiomas o clústeres de ruido que debemos limpiar.
- **Explicabilidad**: Mostrar de manera visual las capacidades del modelo.

¿Para qué **NO** debemos usar estas técnicas?

- **Calcular la similitud** en base a la distancia de los puntos.
- **Recuperación de información** en base al mapa generado.

¿Preguntas?